ARCHITECTURE DECISION RECORD

ADR 002: Modelagem de dados NoSQL

Status: Proposto

Data: 15/10/2026 **Decisores:** João Pedro Pellegrini

1. Contexto e Problema

O Firestore utiliza NoSQL, ele cobra por um documento lido. O sistema atende várias transportadoras, e os dados de uma não podem aparecer para outra. A modelagem precisa atender às consultas do MVP (buscar entrega pelo XML, listar pendências com aging, ver comprovantes e montar o pacote de faturamento), com importação de XML e reenvio de foto sem duplicação. O nome do recebedor é um dado pessoal

2. Decisão

- **Tenant no caminho:** todas as coleções ficam em tenants/{tenantId}/... (members, shippers, deliveries, packages).
- **ID determinístico:** o ID da entrega é a chave de acesso de 44 dígitos do XML, e o ID do comprovante é o hash SHA-256 da foto.
- **Subcoleções:** proofs (comprovantes) e events (trilha) ficam dentro de deliveries.
- **Desnormalização mínima:** a entrega guarda uma cópia de shipperNome, e o shipperId segue como referência.
- **Aging calculado:** guardamos status e pendingSince, e a faixa de aging é calculada na leitura.
- **Tipos seguros:** valores em centavos (inteiro) e datas em Timestamp UTC.
- **LGPD:** só o nome do recebedor, sem CPF ou telefone, e nada de dado pessoal em logs.
- **Versionamento:** firestore.rules e firestore.indexes.json ficam no repositório.

3. Consequências

Positivas	Negativas / Riscos
• O isolamento de tenant vira regra de caminho, fácil de proteger nas Security Rules e de testar (403/404). • Importação e reenvio de foto são idempotentes por construção. • As consultas do MVP são atendidas sem JOIN e com poucas leituras. • O aging fica sempre correto, sem job agendado.	• Dados copiados (shipperNome) podem ficar desatualizados e exigem atualização. • Cada consulta composta exige índice criado e versionado. • Dashboards que varrem muitas entregas geram custo de leitura. • Não há migração automática de esquema, então mudanças exigem script.